TDAD: Test-Driven Agentic Development – Reducing Code Regressions in AI Coding Agents via Graph-Based Impact Analysis

Pepe Alonso
Universidad ORT Uruguay
pepe@berkeley.edu

Abstract

AI coding agents can resolve real-world software issues, yet they frequently introduce regressions, breaking tests that previously passed. Current benchmarks focus almost exclusively on resolution rate, leaving regression behavior under-studied. This paper presents TDAD (Test-Driven Agentic Development), an open-source tool and benchmark methodology that combines abstract-syntax-tree (AST) based code–test graph construction with weighted impact analysis to surface the tests most likely affected by a proposed change. Evaluated on SWE-bench Verified with two local models (Qwen3-Coder 30B on 100 instances and Qwen3.5-35B-A3B on 25 instances), TDAD's GraphRAG workflow **reduced test-level regressions by 70%** (6.08% → 1.82%) and **improved resolution from 24% to 32%** when deployed as an agent skill. A surprising finding is that test-driven development (TDD) prompting *alone* increased regressions (9.94%), revealing that smaller models benefit more from *contextual* information (which tests to verify) than from *procedural* instructions (how to do TDD). I additionally demonstrate an autonomous auto-improvement loop that raised resolution from 12% to 60% on a 10-instance subset with 0% regression. These findings suggest that for AI agent tool design, surfacing contextual information outperforms prescribing procedural workflows. All code, data, and logs are publicly available.

1 Introduction

Large-language-model (LLM) based coding agents have demonstrated impressive ability to resolve real-world software issues. On SWE-bench Verified [1], state-of-the-art agents resolve over 70% of GitHub issues from popular open-source repositories. Evaluation, however, centers almost exclusively on *resolution rate*: the fraction of issues for which the agent's patch passes the issue-specific tests. A complementary question, *how many previously-passing tests does the patch break?*, receives far less attention.

Regressions matter in practice. A patch that fixes one bug but breaks three others would be rejected in any code-review process. Yet detecting regressions is fundamentally hard for agents without dependency awareness. An agent faces two options: (a) run *all* tests in the repository, which does not scale to large codebases where test suites take minutes or hours, or (b) run only the tests near the changed files, which misses indirect dependencies where code several modules away breaks due to a shared interface change. Diffs alone do not reveal these transitive relationships. In the baseline experiments, a vanilla agent caused 562 pass-to-pass (P2P) test failures across 100 instances, an average of 6.5 broken tests per generated patch. The severity varies widely: on `astropy-13977`, a single patch broke all 322 P2P tests, while on `django-13089` TDD prompting alone turned 4 failures into 352, a complete regression of the test suite. These are not edge cases; they represent the default behavior of capable agents operating without structural awareness of code–test dependencies. This paper introduces that awareness.

The standard SWE-bench evaluation harness already collects the data needed to measure regressions: the `PASS_TO_PASS` test set records tests that passed before the gold patch and should still pass afterward. Yet this data is not surfaced in leaderboard rankings or agent evaluation reports. Recent evidence reinforces this concern: METR [14] found that roughly half of SWE-bench-passing patches would not be merged by real maintainers, and Ehsani et al. [15] show that CI/CD failures are a leading cause of rejected agent-authored pull requests in practice. I argue that regression rate deserves first-class status alongside resolution rate.

This paper presents TDAD (**T**est-**D**riven **A**gentic **D**evelopment), an open-source tool and benchmark methodology for measuring and reducing regressions in AI coding agents. TDAD builds an AST-derived code–test dependency graph, applies weighted impact analysis, and surfaces the results as a lightweight agent skill: a static test map and a 20-line instruction file. The design prioritizes zero-dependency integration: the agent needs only `grep` and `pytest` at runtime, with no graph database, MCP server, or API calls. The key insight is that agents do not need to be told *how* to do TDD; they need to be told *which tests to check*.

The key findings, evaluated on SWE-bench Verified with two local models:

(1) **70% regression reduction** (100 instances, Qwen3-Coder 30B). TDAD reduced test-level regression rate from 6.08% to 1.82% (562 → 155 P2P failures).
(2) **TDD prompting paradox.** TDD instructions *without* graph context increased regressions to 9.94%, worse than vanilla.
(3) **Resolution improvement as a skill** (25 instances, Qwen3.5-35B-A3B + OpenCode agent). TDAD improved resolution from 24% to 32% and generation from 40% to 68%.
(4) **Autonomous self-improvement.** An auto-improvement loop raised resolution from 12% to 60% (10-instance subset) with 0% regression.

Contributions. (1) An open-source tool for graph-based test impact analysis (`pip install tdad`); (2) a benchmark methodology elevating regression rate to a first-class metric; (3) empirical evidence across two models and agent frameworks; (4) an autonomous auto-improvement loop for iterative tool refinement; (5) all code, data, and 28 experiments released under MIT license.

The remainder of this paper is organized as follows. Section 2 surveys related work on AI coding agents, regression testing, and graph-based code analysis. Section 3 describes TDAD's architecture

and algorithms. Section 4 details the experimental setup, and Section 5 presents results and analysis. Section 6 discusses implications and limitations, and Section 7 concludes with future directions.

2 Related Work

2.1 AI Coding Agents and Benchmarks

SWE-bench [1] evaluates agents on resolving GitHub issues from 12 popular Python repositories, establishing the primary benchmark for AI coding agents. The SWE-bench Verified subset provides human-validated ground truth for 500 instances. SWE-Agent [4] introduces an agent–computer interface optimized for repository navigation, achieving strong results through careful tool design. AutoCodeRover [5] combines code search with spectrum-based fault localization to improve issue resolution. OpenHands [13] provides a unified platform for developing and evaluating coding agents across multiple benchmarks.

The benchmark ecosystem continues to evolve. SWE-smith [19] scales training data by automatically synthesizing task instances from arbitrary Python codebases, producing 50k instances and training open-source models that achieve 40.2% on SWE-bench Verified. SWE-Bench++ [20] extends evaluation to 11 programming languages with 11k instances. SWE-CI [16] shifts focus from one-shot bug fixing to long-term codebase maintenance via continuous integration loops, requiring agents to handle sequences of commits over months.

All of these efforts focus primarily on resolution rate; regression behavior is reported only incidentally, if at all. The SWE-bench harness *does* execute P2P tests, but results are not surfaced in leaderboard rankings. Ehsani et al. [15] study 33k agent-authored pull requests on GitHub and find that CI/CD pipeline failures and regression are among the most common rejection reasons, confirming that resolution alone is an insufficient measure. This work argues that this omission creates a perverse incentive that rewards aggressive patching regardless of side effects.

2.2 Regression Testing

Regression test selection (RTS) and prioritization have a long history in software engineering. Elbaum et al. [3] survey techniques for improving regression testing in CI environments, establishing that even simple selection strategies can dramatically reduce test execution time. Legunsen et al. [6] evaluate static RTS at scale, finding that class-level dependency tracking achieves good precision. Gligoric et al. [7] propose dynamic file-level dependency tracking via filesystem monitoring. Chianti [8] performs method-level change impact analysis using call-graph differencing for Java programs.

TDAD adapts these classical ideas to a novel setting: AI coding agents, where changes are generated programmatically and must be validated *before* submission. Unlike traditional RTS, which optimizes developer workflow, TDAD uses impact analysis to *inform the agent* about which tests are at risk, enabling self-correction before committing.

2.3 Graph-Based Code Analysis

Code property graphs [9] unify ASTs, control-flow graphs, and program-dependence graphs for vulnerability detection.

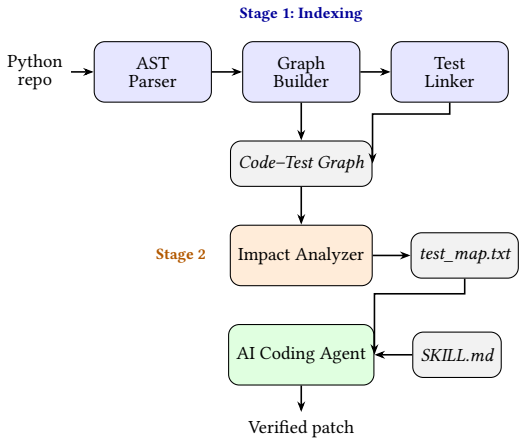

Figure 1: TDAD pipeline. Stage 1 indexes the repository into a code–test graph. Stage 2 exports impacted tests as a static test map. The agent uses `grep` on the test map to find tests to verify.

GraphRAG [2] demonstrates that graph-structured retrieval outperforms flat vector search for complex reasoning. GRACE [17] constructs a multi-level code graph (file structure, AST, call graph, class hierarchy) for repository-aware code completion, achieving 8% improvement over prior graph-based RAG baselines. TDAD applies a similar graph-first principle but targets a different task: rather than code completion, it traverses an explicit code–test dependency graph to identify impacted tests with high precision.

2.4 TDD and AI Agents

Test-driven development (TDD) [10] prescribes writing tests before implementation code, creating a tight feedback loop that catches regressions early. Cui [18] introduces a TDD benchmark where test cases serve as both prompt and verification, finding that *instruction following* and in-context learning matter more than general coding proficiency for TDD success, and that performance degrades when instructions are long. This finding foreshadows the TDD prompting paradox observed in the present work (§5.2).

Rehan [11] independently proposes a "Test-Driven AI Agent Definition" framework that compiles agent *prompts* from behavioral specifications via iterative test–refine loops, achieving 97.2% regression safety. While sharing the TDD philosophy and even the TDAD acronym, the two works target different problems: Rehan validates TDD for agent *behavior compliance*; this paper validates it for agent-generated *code patches*.

3 System Design

3.1 Architecture Overview

Figure 1 shows the TDAD pipeline. Stage 1 (*indexing*) parses a Python repository and builds a code–test dependency graph. Stage 2 (*impact analysis*) identifies tests affected by changed files and exports a static text file (`test_map.txt`). The agent receives this file plus a 20-line skill definition; no MCP server, API calls, or graph database is required at runtime.

Table 1: Graph schema: node and edge types.

Type	Entity	Key attributes
Node types		
File	Python source file	path, content_hash
Function	Top-level function	name, file, lines, signature
Class	Class definition	name, file, bases
Test	Test function/method	name, file, is_test
Edge types		
CONTAINS	File → Function/Class	structural containment
CALLS	Function → Function	static call resolution
IMPORTS	File → File	import tracking
TESTS	Test → Function/Class	test–code linkage
INHERITS	Class → Class	base-class relationship

3.2 Graph Schema

The graph uses four node types and five edge types (Table 1). Nodes represent structural units; edges capture static relationships from AST analysis.

3.3 Indexing Pipeline

The indexer has three components, each producing a specific subset of graph elements.

AST Parser. Parses each Python file using the standard-library `ast` module. Extracts: (a) function definitions with signatures, line ranges, and docstrings; (b) class definitions with base classes and methods; (c) import statements; and (d) call targets within function bodies, resolved via a recursive visitor that handles both simple names and attribute chains. Test files are identified by naming conventions: `test_*.py` and `*_test.py` for files, `test_*` for functions, and `Test*` for classes.

Graph Builder. Populates nodes and edges. Each file produces a File node and child Function/Class nodes connected via CONTAINS edges. Call targets are resolved using module-scoped name resolution, creating CALLS edges. Import statements produce IMPORTS edges; class inheritance produces INHERITS edges.

Test Linker. Creates TESTS edges connecting test nodes to the code they exercise. This is the most critical component, as Python projects use diverse conventions for organizing tests. Three strategies in priority order: (1) *naming conventions* (`test_foo.py` → `foo.py`, including underscore-prefixed variants common in scientific Python); (2) *prefix matching* (progressive truncation of test-file stems to find the best source-file match); (3) *directory proximity* for disambiguation when a stem matches multiple source files. For monolithic test modules (e.g., Django's `tests.py`), a dedicated proximity algorithm maps to source files in the nearest non-test ancestor.

3.4 Impact Analysis

Given changed files, four strategies run in parallel (Table 2) and scores are merged:

$$\text{score} = (1 - c_w) \cdot w_{\text{strategy}} + c_w \cdot \text{confidence} \tag{1}$$

Table 2: Impact analysis strategies and base weights (balanced profile).

Strategy	Weight	Description
Direct	0.95	Directly tests changed code
Transitive	0.70	1–3 call-chain hops to changed code
Coverage	0.80	File-level dependency
Imports	0.50	Imports changed files

where $c_w = 0.3$ is the confidence weight and confidence $\in [0, 1]$ reflects link strength (1.0 for direct TESTS edges, 0.56 for transitive call chains, 0.5 for coverage, 0.45 for imports). When a test appears via multiple strategies, only the highest score is kept. Tests are selected in three tiers: high (≥ 0.8), medium (0.5–0.8), and low (< 0.5), up to a configurable maximum (default: 50). Three weight profiles are available: *conservative* (favors precision), *balanced* (default), and *aggressive* (favors recall).

3.5 Agent Integration: TDAD as a Skill

TDAD integrates with agents through two static artifacts. **test_map.txt** maps source files to test files (one line per mapping, `grep`-able). **SKILL.md** is a 20-line definition: (1) fix the bug, (2) `grep test_map.txt` for related tests, (3) run them and fix failures. This brevity proved critical: through the auto-improvement loop (§5.5), I found that simplifying SKILL.md from 107 to 20 lines alone quadrupled resolution from 12% to 50%.

3.6 Backend Architecture

TDAD originally used Neo4j with Docker for graph storage. Through iterative development, I migrated to a **NetworkX** in-memory backend as the default, eliminating the Docker dependency entirely. Installation is now `pip install tdad` with zero external requirements. Graph persistence uses pickle (`.tdad/graph.pkl`). Neo4j remains available via `TDAD_BACKEND=neo4j` for large-scale deployments. Both backends expose the same `GraphDB` interface.

4 Experimental Setup

4.1 Benchmark

The evaluation uses SWE-bench Verified [1], a human-validated subset of SWE-bench containing 500 GitHub issues from 12 popular Python repositories (Django, scikit-learn, sympy, matplotlib, astropy, Flask, pytest, and others). Each instance specifies: (a) a GitHub issue description, (b) a repository snapshot at issue creation time, (c) `FAIL_TO_PASS` tests that the gold patch fixes, and (d) `PASS_TO_PASS` tests that should remain passing. Phase 1 uses 100 instances (first 100 in canonical order); Phase 2 uses 25 instances selected for diversity.

4.2 Models and Agents

Two experimental phases were conducted (Table 3):

Phase 1 (100 instances) uses **Qwen3-Coder 30B** [12] (Q4_K_M 4-bit quantization) served via llama.cpp on consumer hardware, with a 32K-token context window and temperature 0 for deterministic outputs. Each instance was given up to 15 minutes for

Table 3: Experimental configurations across both phases.

Phase	Config	Description
1	Vanilla	Default prompt, no TDD or graph
1	TDD Prompt	Adds TDD workflow instructions
1	GraphRAG+TDD	TDD + SKILL.md + test_map.txt
2	Baseline	OpenCode agent, no TDAD skill
2	TDAD Skill	OpenCode + TDAD skill + NetworkX

patch generation. This phase evaluates regression reduction across three prompt configurations. I chose a smaller, locally-run model for three reasons: (1) to demonstrate that TDAD's benefits are not contingent on frontier-scale models; (2) to enable fully reproducible experiments without API costs; and (3) to stress-test the approach where every context token matters.

Phase 2 (25 instances) uses **Qwen3.5-35B-A3B** (4-bit quantization, mixture-of-experts with 3B active parameters) served via MLX on Apple Silicon, using the **OpenCode v1.2.24** coding agent. This phase evaluates TDAD deployed as a reusable agent skill with the NetworkX backend, using a different model, quantization framework, and agent from Phase 1 to test generalization of the approach.

Baseline context. For reference, Qwen3-Coder 30B achieves ∼52% on SWE-bench Verified with full-featured scaffolds (SWE-Agent, 100+ turns), and Qwen3.5-35B-A3B achieves 69.2% as reported by Qwen. The lower baselines in this work (31% and 24%) reflect 4-bit quantization, consumer hardware, shorter context windows, and simpler agent scaffolds—conditions chosen to stress-test TDAD under resource constraints.

4.3 Evaluation Protocol

Patches are evaluated using the SWE-bench Docker harness, which for each instance: (1) checks out the repository at the issue's base commit; (2) applies the agent's patch; (3) runs fail-to-pass (F2P) tests (resolution); (4) runs P2P tests (regression detection). Each instance is evaluated in an isolated Docker container. Empty patches are excluded from P2P evaluation.

Four metrics are reported:

- **Resolution rate:** fraction of instances where all F2P tests pass.
- **Generation rate:** fraction producing a non-empty patch.
- **Test-level regression rate:** total P2P failures / total P2P tests across all instances. This is the primary regression metric because it distinguishes a patch breaking 1 test from one breaking 100.
- **Instance-level regression rate:** fraction of generated patches with ≥ 1 P2P failure.

5 Results and Analysis

5.1 Phase 1: Regression Reduction (100 Instances)

Table 4 presents the Phase 1 results with Qwen3-Coder 30B.

GraphRAG+TDD achieved a **72% reduction** in P2P failures (562 → 155) and a **70% reduction in regression rate** (6.08% → 1.82%). Compared to TDD-only, the reduction is even larger: 81% fewer failures (799 → 155). Catastrophic regressions (all P2P tests failing) dropped from 3 in vanilla and 5 in TDD-only to just 1 in GraphRAG+TDD.

Table 4: Phase 1 results: SWE-bench Verified, 100 instances, Qwen3-Coder 30B. ↓ = lower is better.

Metric	Vanilla	TDD Prompt	GraphRAG + TDD
Resolution Rate	**31%**	**31%**	29%
Generation Rate	**86%**	75%	74%
Total P2P Tests	9,245	8,040	8,536
P2P Failures ↓	562	799	**155**
Test Regr. ↓	6.08%	9.94%	**1.82%**
Inst. Regr. ↓	**30.2%**	33.3%	33.3%
Catastrophic* ↓	3	5	**1**

*Instances where all P2P tests failed.

Table 5: Selected instances with severe regressions. Numbers = P2P failures / total. "—" = empty patch (not evaluated).

Instance	Vanilla	TDD	GraphRAG
astropy-13977 (322)	322/322	4/322	12/322
astropy-8872 (80)	80/80	80/80	—
django-13089 (352)	4/352	352/352	—
django-11532 (148)	—	148/148	—
django-11299 (103)	—	103/103	—

Resolution decreased modestly (−2 pp), driven by a higher empty-patch rate (26% vs. 14%); the agent abstained more often when the test map indicated risk. Among patches that were actually generated, GraphRAG was no less likely to resolve the issue.

Note on denominators and instance-level rates. The three configurations produced different numbers of non-empty patches (86, 75, and 74 respectively), yielding different P2P test pools (9,245 vs. 8,040 vs. 8,536 total tests). The instance-level regression rates (30.2%, 33.3%, 33.3%) appear to contradict the test-level result; however, in absolute terms GraphRAG produced *fewer* regressing instances (25 of 74) than vanilla (26 of 86). The higher *rate* is a denominator artifact of GraphRAG's lower generation rate. I report both metrics for transparency but emphasize test-level regression rate as primary because it captures regression *severity*: a patch breaking 322 tests is qualitatively different from one breaking 1.

Table 5 illustrates regression severity on specific instances. On `astropy-13977` (322 P2P tests), vanilla failed all 322 while GraphRAG failed only 12. On `django-13089`, TDD prompting caused total failure (352/352) while vanilla failed only 4. These cases show that TDD prompting without localization can be *catastrophically* worse than doing nothing.

5.2 The TDD Prompting Paradox

TDD prompting *without* graph context increased regressions (6.08% → 9.94%), producing 42% more P2P failures than vanilla. Two factors explain this:

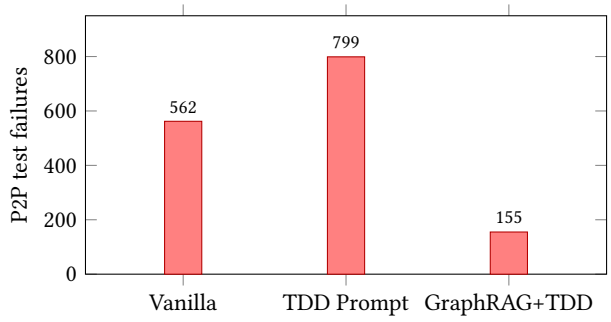

Figure 2: Total P2P failures per approach (Phase 1). GraphRAG+TDD reduces failure count by 72% vs. vanilla and 81% vs. TDD-only.

Verbose prompts hurt small models. The TDD prompt consumed context tokens with procedural instructions, pushing out repository context that the 30B model needed for accurate changes.

Ambition without localization. TDD-prompted agents attempted more ambitious fixes, touching more files. Without graph-based knowledge of *which* tests to verify, this ambition caused collateral damage. The five catastrophic regressions in TDD-only (vs. three in vanilla) confirm this. GraphRAG counteracted both effects: the 20-line SKILL.md freed context tokens, and the test map focused attention on tests at risk.

Ablation evidence. Prior experiments isolated the respective contributions. Shortening the prompt *without* graph context (from 119 to 49 lines) decreased resolution from 30% to 20%, confirming that prompt brevity alone does not explain GraphRAG's improvement. Conversely, doubling prompt length without graph context (49-line vanilla vs. 119-line TDD) produced identical 31% resolution, showing that additional procedural text neither helped nor hurt in isolation. The improvement requires graph-derived context: a short prompt *plus* the test map.

Practical implication: for smaller models, context (which tests to check) outperforms procedure (how to do TDD).

5.3 Resolution vs. Regression Trade-off

The modest 2-percentage-point resolution drop (31% → 29%) represents an intentional conservative bias. GraphRAG agents produced more empty patches (26% vs. 14%) when the test map indicated high regression risk; the agent "knew what it didn't know" and abstained.

Notably, in our experiments no approach produced patches that were simultaneously resolved *and* regressed ("Resolved w/ regr." = 0 across all configs). This suggests that correct fixes and regressions are largely disjoint failure modes for this model: patches either solve the problem cleanly or cause damage, but rarely both. This observation has implications for agent design: mechanisms that suppress harmful patches (like TDAD's test verification step) incur minimal cost to resolution quality.

Table 6: Phase 2 results: SWE-bench Verified, 25 instances, Qwen3.5-35B-A3B + OpenCode agent.

Metric	Baseline	TDAD Skill	Delta
Resolved	6/25 (24%)	**8/25 (32%)**	+8pp
Generated	10/25 (40%)	**17/25 (68%)**	+28pp
Res. of generated	6/10 (60%)	8/13 (62%)	+2pp
Empty patches	15	**8**	−7
Regression Rate	0%	0%	0pp

5.4 Phase 2: TDAD as an Agent Skill (25 Instances)

After the Phase 1 findings and an auto-improvement loop (§5.5), I packaged TDAD as a reusable skill with a NetworkX backend (no Docker) and evaluated it with a different model and agent framework. Table 6 shows the results.

TDAD as a skill improved resolution by +8 percentage points (24% → 32%) and generation by +28 pp (40% → 68%). Four previously-empty instances were resolved with the TDAD skill, while two baseline resolutions were lost due to model non-determinism.

The improvement mechanism differs from Phase 1: rather than reducing regressions (both had 0% on this smaller instance set), the test map helped the agent *generate more patches* by providing structural context. The code–test graph served a dual purpose: identifying tests to verify (reducing regressions, as in Phase 1) and mapping codebase structure (improving navigation and generation, as in Phase 2).

5.5 Auto-Improvement Loop

Inspired by Karpathy's `autoresearch` framework,[1] I built an outer loop that autonomously improves TDAD's components through iterative agent-driven modification and evaluation.

The loop (Algorithm 1) operates as follows. Each iteration: (1) a Claude Code agent receives the full experiment history and makes one focused change to TDAD's source files (SKILL.md, `impact.py`, `ast_parser.py`, etc.); (2) unit tests gate acceptance: if they fail, the change is immediately reverted; (3) a benchmark evaluation (5–25 SWE-bench instances) measures generation and resolution rates; (4) results are compared to the best known scores: improvements update the "best" snapshot, regressions trigger rollback, and lateral moves (same resolution) are kept to allow exploration. Integrity safeguards prevent gaming: the evaluation script is checksummed (SHA-256) and set read-only, and 5 consecutive reverts trigger a mandatory restore to the best snapshot.

Over 15 iterations (Table 7), 4 changes were accepted (27% rate). Generation rose from 28% to 80% (+52 pp) and resolution from 12% to 60% (+48 pp), with 0% regression throughout all iterations.

The most impactful single change (iteration 1) was simplifying SKILL.md from 107 lines of detailed 9-phase TDD instructions to 20 lines of concise guidance: fix, grep, verify. This alone quadrupled resolution (12% → 50%), confirming that the small model

[1] https://github.com/karpathy/autoresearch

Algorithm 1 Auto-improvement loop.

```
Require: snapshot S_best, evaluator E, max iters N
for i = 1 to N do
    S_pre ← snapshot current files
    Invoke agent: "make one improvement"
    if no files changed then continue
    end if
    if unit tests fail then
        Restore(S_pre); continue
    end if
    r ← E(current files)
    if r.resolution > S_best.resolution then
        S_best ← snapshot current files
    else if r.resolution < S_best.resolution then
        Restore(S_best)
    end if                                  ▷ Lateral: keep as-is
end for
```

Table 7: Auto-improvement loop (15 iterations, 10-instance eval). Only accepted iterations shown; 11 were reverted.

Iter	Changed	Gen%	Res%	Key Change
1	SKILL.md	50	50	Simplified 107→20 lines
5	impact.py	70	60	Static test-map export
12	impact.py	70	60	Path proximity scoring
13	impact.py	80	60	Import-based mappings
Before loop		28	12	—
After loop (best)		**80**	**60**	—
Vanilla baseline		40	24	—

(3B active parameters) could not effectively use the verbose workflow. Subsequent iterations improved test-mapping heuristics: iteration 5 added static `test_map.txt` export, iteration 12 introduced directory-proximity disambiguation, and iteration 13 added import-based fallback matching.

The 11 rejected iterations provide equally valuable signal: attempts to expand the AST parser, restructure the graph builder, or make SKILL.md more prescriptive all decreased resolution. The loop converged by iteration 5, with later iterations showing diminishing returns.

6 Discussion

6.1 Context Over Procedure

These results have implications for how developers design tools for AI coding agents. The dominant approach, crafting detailed prompts with step-by-step instructions, may be counterproductive for smaller models. Instead, TDAD's success suggests that *surfacing the right information* (which tests are at risk, which files are related) matters more than *prescribing workflow* (write test first, then implement, then refactor).

This finding aligns with broader observations in retrieval-augmented generation: context quality is the primary determinant of output quality [2]. Tool designers for AI agents should prioritize *information density* over *procedural completeness*: short, factual context that fits within tight context windows outperforms verbose instructions that crowd out useful information.

6.2 Regression as a First-Class Metric

Current AI coding benchmarks create a perverse incentive: agents are rewarded for resolution regardless of collateral damage. In practice, a patch that resolves one issue but introduces three regressions has negative value. METR [14] demonstrates this gap concretely: when maintainers of scikit-learn, Sphinx, and pytest reviewed 296 SWE-bench-passing patches, roughly half would not have been merged, with regression and code quality among the top rejection reasons. I advocate for reporting regression rate alongside resolution rate as standard practice. The data is already collected by the SWE-bench Docker harness; it simply needs to be surfaced in evaluation reports and leaderboard rankings.

A simple composite metric could capture both dimensions: net score $=$ resolution rate $- \alpha \cdot$ regression rate, where $\alpha > 1$ reflects the asymmetric cost of regressions versus missed fixes.

6.3 Limitations

- **Scale and statistical power.** 100 instances (Phase 1) and 25 instances (Phase 2) without formal significance tests. The Phase 1 effect size is large (72% reduction, 562 vs. 155 failures), suggesting practical significance, but smaller effects could be noise at this sample size. Results should be confirmed on the full 500-instance set.
- **Two models.** Both are smaller local models. Frontier models may not exhibit the same TDD-prompting paradox.
- **Python only.** Extension to other languages requires language-specific parsing (Tree-sitter could unify this).
- **Static analysis.** Cannot capture dynamic dispatch, monkey-patching, or runtime-generated code.
- **Auto-loop scale.** Evaluated on only 10 instances per iteration.

6.4 Threats to Validity

Internal validity. Configurations were run sequentially rather than interleaved, so time-dependent factors (e.g., thermal throttling on consumer hardware) could systematically favor earlier runs. This risk is mitigated by temperature 0 for deterministic decoding and Docker isolation for evaluation. The auto-improvement loop used a fixed 10-instance evaluation set; overfitting to those instances cannot be ruled out. However, two factors mitigate this concern: (1) the accepted changes were structural improvements to test-linking heuristics (directory proximity, import-based fallback) and prompt simplification, none of which encode instance-specific knowledge; and (2) the Phase 2 evaluation on a disjoint 25-instance set with a different model and agent confirmed that the optimized tool generalizes beyond the loop's training set.

External validity. SWE-bench Verified draws exclusively from well-maintained, heavily-tested Python repositories. Projects with sparse test suites, non-Python codebases, or monorepo structures may respond differently to graph-based impact analysis. The two models used (Qwen3-Coder 30B and Qwen3.5-35B-A3B) are both smaller, locally-run models; frontier-scale models with longer context windows may not exhibit the same TDD-prompting paradox, as

verbose instructions would consume a smaller fraction of available context.

Construct validity. Test-level regression rate weights all P2P failures equally, regardless of test importance or business impact: a failing unit test for a utility function counts the same as a failing integration test for a critical API endpoint. Instance-level regression rate partially addresses this by treating any regression as binary, but neither metric captures severity. Additionally, the P2P test set is defined by the SWE-bench harness and may not cover all code paths affected by a patch.

7 Conclusion

This paper presented TDAD, an open-source tool and methodology for reducing regressions in AI coding agents via graph-based test impact analysis. Across two models and agent frameworks on SWE-bench Verified, TDAD achieved a **70% reduction in test-level regressions** (Phase 1) and **+8pp resolution improvement** when deployed as an agent skill (Phase 2). The experiments revealed a TDD prompting paradox: procedural instructions *increased* regressions for smaller models, while graph-derived context reduced them. An auto-improvement loop demonstrated self-improving tool design, raising resolution from 12% to 60% with 0% regression.

Future work includes extending TDAD to multiple languages via Tree-sitter, integrating dynamic coverage data to improve impact precision, and evaluating with frontier models (e.g., Claude Opus 4.6, GPT-5.4) to test whether the TDD prompting paradox persists at larger scale. I advocate for the community to adopt composite metrics that jointly capture resolution and regression, ensuring agents are evaluated on net contribution.

8 Data Availability

All artifacts are publicly available:

- **Repository:** https://github.com/pepealonso95/TDAD
- **Install:** `pip install tdad` (zero dependencies beyond NetworkX)
- **Experiment log:** `EXPERIMENTS.md` (28 experiments, EXP-001 through EXP-028)
- **All patches, evaluation logs, and auto-loop history** archived in the repository under MIT license.

Use of Generative AI. Generative AI tools were used to assist in writing implementation code, organizing experimental logs, summarizing implementation details, and improving manuscript wording. The author performed the experimental design, conducted the work, interpreted the results, and verified the final content.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. NeurIPS*, 2024.

[2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[3] S. Elbaum, G. Rothermel, and J. Penix. Techniques for improving regression testing in continuous integration development environments. In *Proc. FSE*, 2014.

[4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proc. NeurIPS*, 2024.

[5] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proc. ISSTA*, 2024.

[6] O. Legunsen, F. Hariri, A. Shi, Y. Lu, L. Zhang, and D. Marinov. An extensive study of static regression test selection in modern software evolution. In *Proc. FSE*, 2016.

[7] M. Gligoric, R. Majumdar, R. Sharma, L. Eloussi, and D. Marinov. Practical regression test selection with dynamic file dependencies. In *Proc. ISSTA*, 2015.

[8] X. Ren, F. Shah, F. Tip, B. G. Ryder, and O. Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proc. OOPSLA*, 2004.

[9] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck. Modeling and discovering vulnerabilities with code property graphs. In *Proc. IEEE S&P*, 2014.

[10] K. Beck. *Test-Driven Development: By Example.* Addison-Wesley, 2003.

[11] T. Rehan. Test-driven AI agent definition (TDAD): Compiling tool-using agents from behavioral specifications. *arXiv preprint arXiv:2603.08806*, 2026.

[12] R. Cao, M. Chen, J. Chen, et al. Qwen3-Coder-Next technical report. *arXiv preprint arXiv:2603.00729*, 2026.

[13] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. In *Proc. ICLR*, 2025.

[14] METR. Many SWE-bench-passing PRs would not be merged into main. Technical note, March 2026. https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/

[15] R. Ehsani, S. Pathak, S. Rawal, A. Al Mujahid, M. M. Imran, and P. Chatterjee. Where do AI coding agents fail? An empirical study of failed agentic pull requests in GitHub. *arXiv preprint arXiv:2601.15195*, 2026.

[16] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. *arXiv preprint arXiv:2603.03823*, 2026.

[17] X. Wang, B. Wang, C. Zhi, J. Han, X. Zhao, J. Yin, and S. Deng. GRACE: Graph-guided repository-aware code completion through hierarchical code fusion. *arXiv preprint arXiv:2509.05980*, 2025.

[18] Y. Cui. Tests as prompt: A test-driven-development benchmark for LLM code generation. *arXiv preprint arXiv:2505.09027*, 2025.

[19] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. SWE-smith: Scaling data for software engineering agents. In *Proc. NeurIPS D&B*, 2025.

[20] L. Wang, L. Ramalho, A. Celestino, et al. SWE-Bench++: A framework for the scalable generation of software engineering benchmarks from open-source repositories. *arXiv preprint arXiv:2512.17419*, 2025.